



UITs
UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES

SOFTWARE PROJECT DESIGN AND DEVELOPMENT LAB

Software Documentation

Course Code: CSE-416

Project: Homemade Food Delivery Management System

SUBMITTED TO

Mr. Al-Imtiaz
Associate Professor & Head, Department of CSE
University of Information Technology & Sciences (UITs)

SUBMITTED BY

Md. Mefazur Rahman (0432310005101035)
Md. Shoyaif Rahman (0432310005101050)
Batch: 53
Semester: Spring (7th)
Section: 7A2
Date of Submission: 13/05/2026

Table of Contents

Table of Contents	1
Introduction	2
User Manual	2
1. Getting Started: Account Registration & Login	2
2. Seller Features & Navigation	2
3. Customer Features & Navigation	3
4. Admin Dashboard	3
5. System Constraints	3
System Requirements.....	4
1. Technology Stack	4
2. Software & Environment Requirements	4
3. Functional Requirements (Implemented Features)	4
A. User Authentication & Session Management.....	4
B. Food Posting & Approval	5
C. Ordering & Payment.....	5
D. Delivery Management	5
4. Non-Functional Requirements	5
Design and Methodology	6
1. System Architecture	6
2. System Modules	6
3. Development Methodology	6
4. Diagrams.....	7
A. Use Case Diagram	7
B. Activity Diagram.....	7
C. Class Diagram.....	8
D. Swimlane Diagram	9
Conclusion and Justification.....	11
1. Conclusion	11
2. Justification: Course Outline Alignment	11

Introduction

Welcome to the Homemade Food Delivery Management System, a comprehensive web platform designed to connect home cooks (Sellers) with food lovers (Customers) through a seamless and transparent ordering ecosystem. The platform serves all stakeholders — Sellers, Customers and Administrators — through a single integrated system.

Sellers can register, create food listings, and manage their homemade food items, which are subject to Admin approval before becoming visible to the public. Customers can browse approved food items, add them to a favorite and confirm order through secure online payment. Once an order is placed, the chef can ensure to deliver the food.

The system prioritizes transparency, security, and usability. With role-based access control ensuring every user sees only what they need, the platform maintains data integrity at every step — from food post approval to payment processing and delivery confirmation.

User Manual

This user manual provides step-by-step guidance for each type of user interacting with the Homemade Food Delivery Management System.

1. Getting Started: Account Registration & Login

All users (Sellers, Customers) must register and log in through the platform's authentication system.

- Visit the platform's homepage and click Register.
- Select your role: Customer, Seller.
- Fill in your name, email, photo, address, and create a password.
- Submit the form. Your account will be created with role-based access permissions.
- To log in, enter your registered email and password on the Login page.
- Admins are pre-configured by the system owner and log in via the Admin Login portal.

2. Chef Features & Navigation

After logging in as a Seller, you will have access to a dedicated dashboard with the following capabilities:

- **Create Meal:** Navigate to the 'My Food Items' section and click 'Add New Item'. Fill in the food name, description, price, and quantity, then submit for Admin approval.
- **Update Food Item:** Select an existing item and click 'Edit' to update its details.
- **Delete Food Item:** Remove a food item from your listings using the 'Delete' button.
- **Meal Orders:** Monitor incoming orders for your food items from the 'Meal Orders' section.
- **My Profile:** Show the profile details.

3. Customer Features & Navigation

After logging in as a Customer, you can browse, order, and track homemade food:

- **Browse Food:** View all approved food listings on the home/browse page. Use filters (category, price, seller) to narrow your search.
- **Add to Favorite:** Click 'Add to favorite' on any food item.
- **Place Order:** Go to your cart and click 'Place Order'. Review the order summary.
- **Payment on Stripe:** Online Payment (secure gateway). Complete the payment online.
- **Track Order:** After placing an order, visit 'My Orders' to see live delivery status updates — Pending, Rejected, Delivered.

5. Admin Dashboard

The Admin interface is the control center for the entire platform:

- **Approve or Reject Food Posts:** Review pending food submissions from Sellers and approve or reject them with optional feedback.
- **Manage Users:** View, activate, deactivate, or remove user accounts (Sellers, Customers, Riders).
- **Monitor Orders:** View all active and past orders with full order details, payment status, and delivery progress.
- **System Reporting:** Access summary dashboards for order volume, active users, and delivery performance.

6. System Constraints

- Sessions are tied to user role and credentials — unauthorized access to other roles' dashboards is blocked.
- Online payments are processed through a secure payment gateway.
- Fraud user can't order food, make payment, create a meal.

System Requirements

1. Technology Stack

- **Front-End:** React.js, Tailwind CSS, HTML5, and JavaScript for building a modern, responsive, and cross-browser compatible user interface.
- **Back-End:** Node.js and Express.js for handling server-side logic, RESTful APIs, authentication, and role-based access control.
- **Database:** MongoDB for storing user data, food items, meal orders, payments, and other application information efficiently.
- **Authentication & Security:** Firebase Authentication and JWT (JSON Web Token) for secure login, protected routes, and user role management.
- **Payment Integration:** Stripe payment gateway integration for secure and seamless online transactions.
- **Deployment & Hosting:** Netlify is used for frontend deployment, while Vercel is used for backend hosting and API deployment.
- **Development Environment:** MERN Stack development using MongoDB, Express.js, React.js, and Node.js with Git & GitHub for version control and collaboration.

2. Software & Environment Requirements

- **Frontend Hosting:** Netlify for deploying and hosting the React.js frontend application.
- **Backend Hosting:** Vercel for hosting the Node.js and Express.js backend APIs.
- **Database Server:** MongoDB Atlas for cloud-based database management and secure data storage.
- **Runtime Environment:** Node.js for running the backend server and managing project dependencies.
- **Web Browser:** Any modern browser such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari supporting JavaScript ES6 and responsive web technologies.
- **Development Tools:** Visual Studio Code, Git, and GitHub for coding, version control, and collaboration.
- **Operating System:** Windows, macOS, or Linux with support for Node.js, MongoDB, and modern web development tools.

3. Functional Requirements (Implemented Features)

A. User Authentication & Session Management

- Multi-role registration and login (Customer, Chef, Admin).
- Role-based access control (RBAC) — each role sees only its authorized features.
- Secure session management tied to user identity and role.

B. Food Posting & Approval

- Chef can add, update, and delete food items with name, description, price, and quantity.
- If Chef is approved for the role then he can upload food, update, and delete food.

C. Ordering & Payment

- Customers can browse approved food, search food, and place orders.
- Payment options: Online Payment through stripe (secure gateway) .
- Order confirmation is generated automatically after successful payment.

D. Delivery Management

- When payment is done then Chef can allow for order
- Customers receive real-time status updates and can track their order.

4. Non-Functional Requirements

- Performance: Supports up to 1,000 concurrent users without degradation.
- Security: Role-based access control with secure session handling and encrypted passwords.
- Usability: Fully responsive website — mobile-friendly and multi-browser compatible.
- Reliability: Regular data backups; safe and error-free transaction processing.
- Scalability: Designed to accommodate growing user base and data volume in the future.

Design and Methodology

1. System Architecture

The Local Chef Bazar platform follows a modern three-tier MERN stack architecture that separates the user interface, application logic, and data management system for better scalability, performance, and maintainability.

- **Presentation Layer (Frontend):** Built using React.js, Tailwind CSS, HTML5, and JavaScript to provide a responsive, user-friendly, and cross-browser compatible interface for customers, chefs, and admins.
- **Application Layer (Backend):** Powered by Node.js and Express.js, this layer manages RESTful APIs, user authentication, JWT-based authorization, role-based access control, payment integration, food item management, and communication between the frontend and database.
- **Data Layer (Database):** MongoDB Atlas is used as the cloud database to securely store and manage user accounts, food items, meal orders, payment information, reviews, and application activity records.
- **Authentication & Security:** Firebase Authentication and JSON Web Tokens (JWT) are implemented for secure login, protected routes, and user role management.
- **Deployment Architecture:** The frontend application is deployed on Netlify, while the backend server and APIs are hosted on Vercel for fast and scalable cloud deployment.

2. System Modules

- **User Management Module:** Handles user registration, login, authentication, profile management, and role-based access control for Admins, Chefs, and Customers.
- **Meal Management Module:** Chefs can create, update, and delete food items, while Admins can approve or reject submitted meal posts.
- **Browsing and Ordering Module:** Customers can browse meals, search and filter food items, view detailed meal information, add items to cart, and place orders easily.
- **Payment Processing Module:** Integrates secure Stripe payment gateway for handling online transactions and payment confirmations.
- **Order Management Module:** Allows Chefs and Admins to monitor meal orders, manage order statuses, and track customer purchases.
- **Admin Dashboard Module:** Provides centralized monitoring, analytics, user management, meal approval controls, and overall platform management features.

3. Development Methodology

The Local Chef Bazar platform was developed using an Iterative Prototyping approach to ensure continuous improvement, feature validation, and user-friendly system design.

- **Requirement Analysis:** Identifying user roles such as Admin, Chef, and Customer, defining system modules, and planning the complete meal ordering and management workflow.
- **UI/UX Design:** Designing a modern, responsive, and user-friendly interface compatible with desktop, tablet, and mobile devices.

- **Database Modeling:** Designing the MongoDB database structure to efficiently manage users, food items, meal orders, payments, and application records.
- **Backend Development:** Developing RESTful APIs and server-side logic using Node.js and Express.js for authentication, role-based access control, meal management, payment integration, and order processing.
- **Frontend Development:** Building dynamic and interactive user interfaces using React.js, Tailwind CSS, and modern JavaScript features.
- **Testing & Validation:** Performing manual testing, functionality testing, integration testing, and responsive design testing to ensure system reliability and performance.
- **Deployment & Maintenance:** Deploying the frontend on Netlify and backend services on Vercel while maintaining version control with GitHub.

4. Diagrams

A. Use Case Diagram

The Use Case Diagram illustrates the interactions between the four primary actors — Customer, Seller, Rider, and Admin — and the core system functions. It shows how each actor participates in actions such as browsing food, placing orders, approving posts, assigning riders, and tracking deliveries.

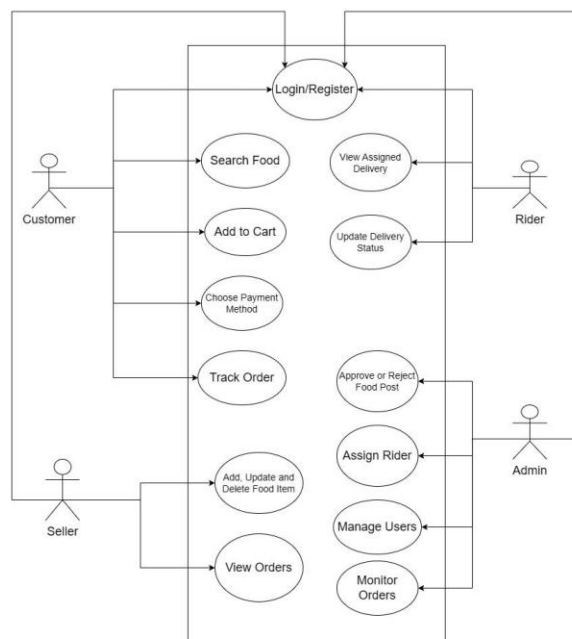


Fig-1: Use Case Diagram

B. Activity Diagram

The Activity Diagram maps the step-by-step flow of the ordering process — from Login/Register through searching food, adding to cart, selecting a payment method, processing payment, assigning a rider, and finally tracking the delivery to completion.

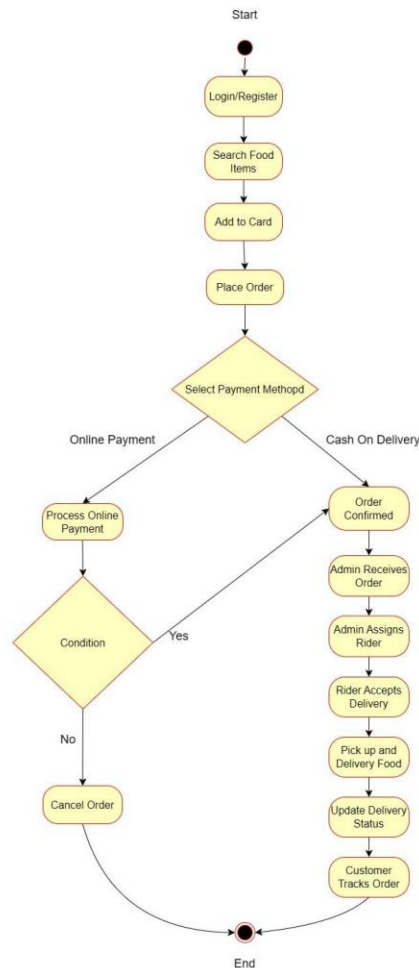


Fig-2: Activity Diagram

C. Class Diagram

The Class Diagram defines the object-oriented structure of the system, showing all entities (User, Admin, Seller, Customer, Rider, Food, Order, Payment) with their attributes, methods, and the relationships between them including inheritance, association, and multiplicity.

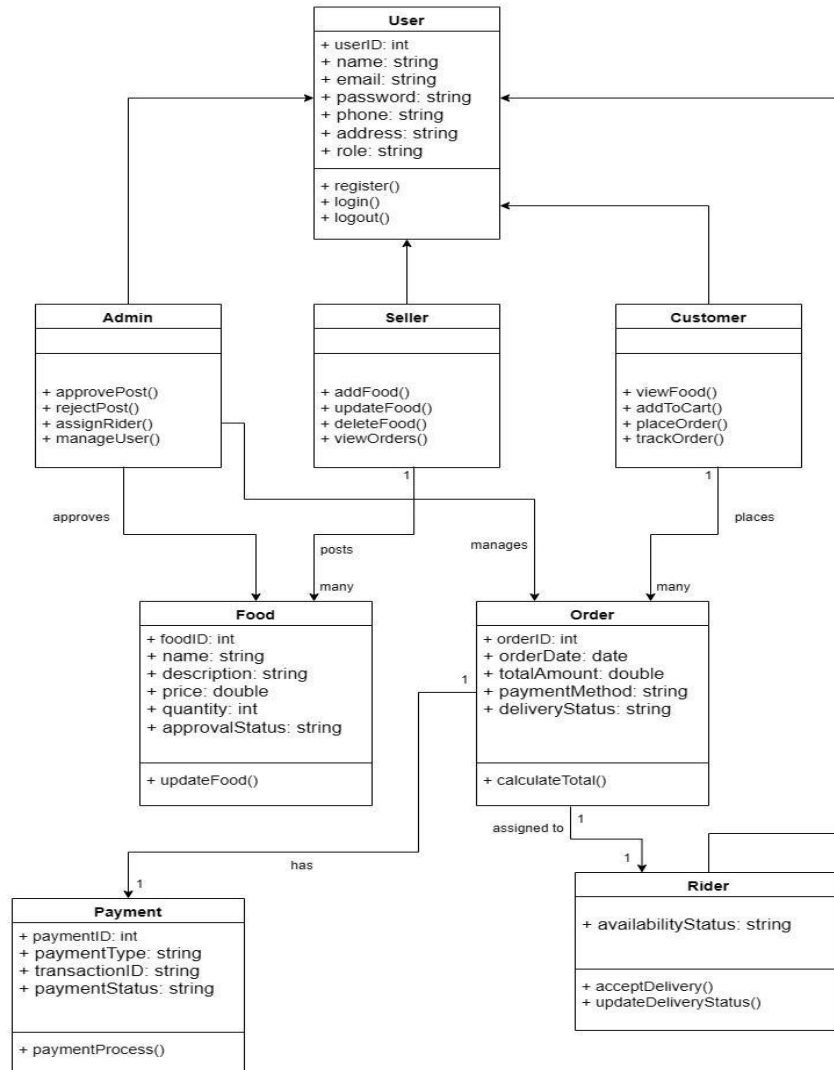


Fig-3: Class Diagram

D. Swimlane Diagram

A Swimlane Diagram is a type of process flowchart that visually separates responsibilities into different 'lanes', where each lane represents a person, team, or system. It shows how tasks move across these lanes, making it clear who is responsible for each step and how different parts of a process interact. The diagram below illustrates the complete order lifecycle across all five stakeholders of the Homemade Food Delivery Management System.

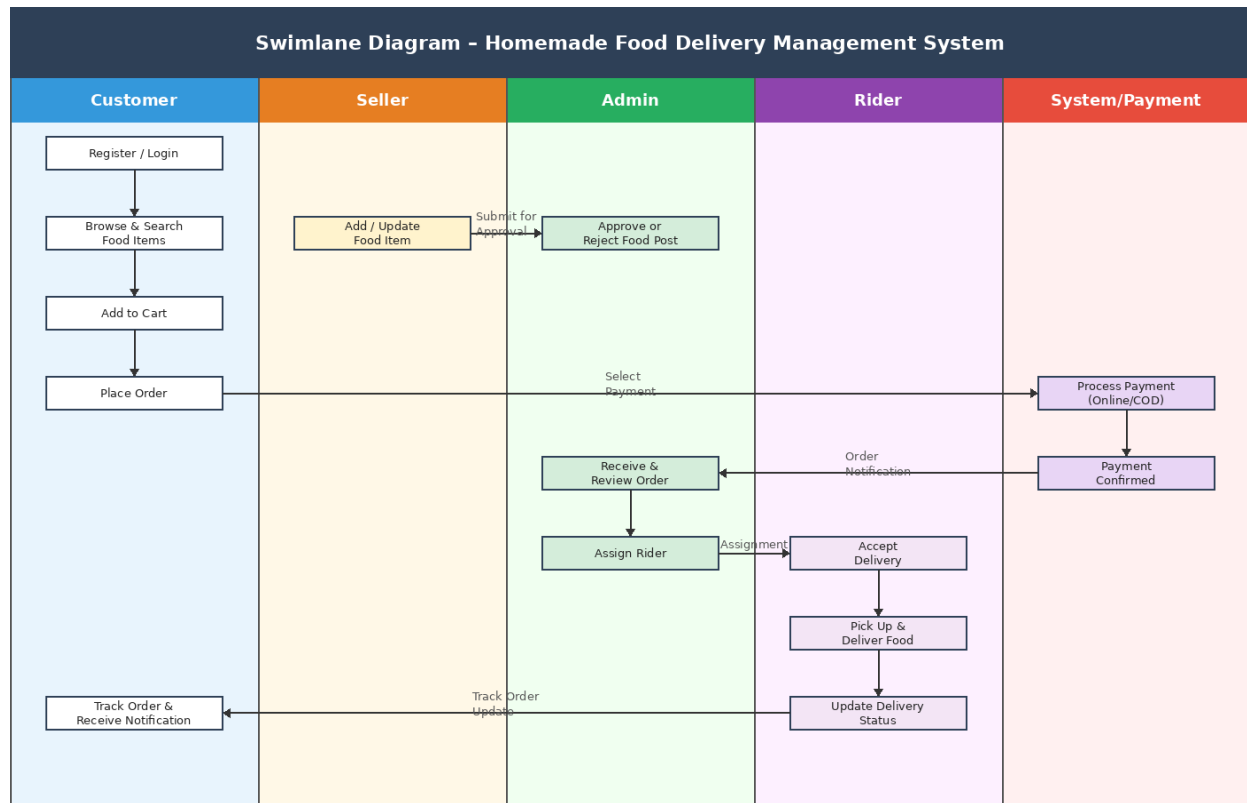


Fig-4: Swimlane Diagram

Conclusion and Justification

1. Conclusion

The Local Chef Bazar platform successfully addresses the growing demand for a reliable, scalable, and community-driven marketplace that connects local chefs with food lovers through a modern MERN stack web application. By implementing role-based access control for Admins, Chefs, and Customers, the system delivers a complete end-to-end solution for managing homemade meal commerce efficiently.

The Admin-controlled meal approval workflow ensures that only verified and quality meal listings are published on the platform, helping maintain content quality and user trust. Secure authentication, protected routes, and Stripe-based online payment integration provide a safe and seamless experience for users.

Customers can browse meals, place orders, manage their purchases, and interact with the platform through a responsive and user-friendly interface. Chefs can manage food items, monitor meal orders, and update their profiles from a dedicated dashboard.

The modular MERN stack architecture using React.js, Node.js, Express.js, and MongoDB ensures high scalability, maintainability, and performance. Frontend deployment through Netlify and backend hosting with Vercel further enhance the platform's accessibility and reliability.

This project demonstrates a strong understanding of the complete software development lifecycle, including requirement analysis, responsive UI/UX design, database modeling, API development, authentication, deployment, testing, and documentation.

2. Justification: Course Outline Alignment

This project was developed in accordance with the standards and requirements defined in the course outline for CSE416 – Software Project Design and Development. The implementation of the Local Chef Bazar platform reflects the practical application of software engineering principles, full-stack web development, database management, system analysis, UI/UX design, testing, and deployment practices.

The following section presents the mapping between the project features and functionalities with the corresponding Course Outcomes (COs) and Program Outcomes (POs) outlined in the CLO document. This mapping demonstrates how the project fulfills academic objectives through requirement analysis, system design, MERN stack implementation, testing, deployment, and documentation.

Outcome	Implementation Justification
CO1: Design software systems (PO3)	Designed the Homemade Food Delivery Management System by collecting and analyzing project requirements. This included developing a comprehensive SRS document with user characteristics, functional requirements, system modules, and UML diagrams (Use Case, Activity, Class Diagram) that define the full architecture of the web platform.
CO2: Apply modern	Applied modern engineering tools and technologies including PHP for backend logic, MySQL for database management, and XAMPP for local server deployment. The project also leverages a secure online payment gateway, role-

Outcome	Implementation Justification
engineering tools (PO5)	based access control mechanisms, and responsive web design principles using HTML5, CSS3, and JavaScript.
CO3: Testing & Quality Assurance (PO9)	Implemented a robust food post approval workflow where the Admin reviews and verifies seller-submitted food items before they become publicly visible. This Review-before-Publish mechanism ensures data integrity and software quality by preventing unverified or inappropriate content from appearing to customers.
CO4: Complex Engineering Solutions (PO12)	Built a real-life complex software solution that addresses the homemade food distribution problem by integrating multi-role user management (Seller, Customer, Rider, Admin), an end-to-end order lifecycle with payment processing (Online and COD), and a rider assignment and delivery tracking system — all within a single cohesive web platform.
CO5: Technical Communication & Teamwork (PO10)	Demonstrated effective communication of software engineering concepts through this comprehensive technical documentation, including system architecture, UML diagrams, a swimlane diagram, and a user manual. The project reflects coordinated teamwork in designing, developing, and presenting a fully functional food delivery management system.